

01b — Model Training on FMA (UNet_Audio_Classifier)

Allena solo `UNet_Audio_Classifier` su FMA e salva risultati.

```
In [1]: # Logger (consistent with setup notebook)
import os
from datetime import datetime
```

```
VERBOSE = os.environ.get('FMA_VERBOSE', '1') == '1'
```

```
def log(msg: str, level: str = 'INFO'):
    if not VERBOSE and level == 'INFO':
        return
    ts = datetime.now().strftime('%H:%M:%S')
    print(f'[{ts}] {level}: {msg}')
```

```
In [2]: # Training config toggles (override via env if needed)
```

```
# Set to '1' to ignore checkpoint and train from scratch
```

```
FMA_FORCE_RETRAIN = os.environ.get('FMA_FORCE_RETRAIN', '0') == '1'
```

```
# Set to '1' to RESUME training from existing checkpoint (continue training)
```

```
FMA_RESUME_FROM_CKPT = os.environ.get('FMA_RESUME_FROM_CKPT', '0') == '1'
```

```
# If '1' and checkpoint exists (and not forcing/resuming), skip training
```

```
SKIP_TRAIN_IF_CKPT = os.environ.get('SKIP_TRAIN_IF_CKPT', '0') == '1'
```

```
# Training hyperparams
```

```
FMA_EPOCHS = int(os.environ.get('FMA_EPOCHS', '80'))
```

```
FMA_PATIENCE = int(os.environ.get('FMA_PATIENCE', '15'))
```

```
# Batch size (reuse your env var if set)
```

```
FMA_BATCH_SIZE = int(os.environ.get('INFER_BATCH_SIZE', os.environ.get('FMA_
```

```
# Regularization / Augmentation
```

```
FMA_LABEL_SMOOTH = float(os.environ.get('FMA_LABEL_SMOOTH', '0.05')) # smal
```

```
FMA_SPEC_AUGMENT = os.environ.get('FMA_SPEC_AUGMENT', '1') == '1'
```

```
FMA_FREQ_MASK_PARAM = int(os.environ.get('FMA_FREQ_MASK_PARAM', '16'))
```

```
FMA_TIME_MASK_PARAM = int(os.environ.get('FMA_TIME_MASK_PARAM', '32'))
```

```
FMA_NUM_MASKS = int(os.environ.get('FMA_NUM_MASKS', '2'))
```

```
log(f"Config -> FORCE_RETRAIN={FMA_FORCE_RETRAIN}, RESUME_FROM_CKPT={FMA_RES
```

[09:02:51] INFO: Config -> FORCE_RETRAIN=False, RESUME_FROM_CKPT=False, SKIP_TRAIN_IF_CKPT=False, EPOCHS=80, PATIENCE=15, BATCH=32, SPEC_AUG=True, LS=0.05

```
In [3]: # Load processed FMA data (clean setup)
import os
# Toggle: set TRAIN_ON_GPU=1 in env to enable GPU; default CPU to avoid CUDA
if os.environ.get('TRAIN_ON_GPU', '0') != '1':
    os.environ['CUDA_VISIBLE_DEVICES'] = '-1'

import pickle, time, numpy as np, pandas as pd, tensorflow as tf
import keras
from keras import layers, models, callbacks
from keras.utils import to_categorical
from pathlib import Path

# Reproducibility and safer TF setup
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
tf.random.set_seed(RANDOM_STATE)

# Enable GPU memory growth if GPUs are present to prevent OOM crashes (no-op)
try:
    gpus = tf.config.list_physical_devices('GPU')
    for gpu in gpus:
        tf.config.experimental.set_memory_growth(gpu, True)
except Exception as e:
    log(f'TF GPU setup warning: {e}', level='WARN')

PROJECT_ROOT = Path(os.getcwd()).resolve().parents[1]
PROCESSED = PROJECT_ROOT/'data'/'processed_fma'
MODELS = PROJECT_ROOT/'models'
REPORTS = PROJECT_ROOT/'reports'
MODELS.mkdir(exist_ok=True); REPORTS.mkdir(exist_ok=True)

# Load arrays (memory-mapped to reduce RAM spikes)
X_train = np.load(PROCESSED/'X_train.npy', mmap_mode='r'); y_train = np.load(P
X_val = np.load(PROCESSED/'X_val.npy', mmap_mode='r'); y_val = np.load(PROCE
X_test = np.load(PROCESSED/'X_test.npy', mmap_mode='r'); y_test = np.load(PF

# Ensure channel dimension exists and time dimension matches across splits
# Shapes expected: (N, n_mels=128, n_frames, 1)
if X_train.ndim == 3:
    X_train = X_train[..., None]
if X_val.ndim == 3:
    X_val = X_val[..., None]
if X_test.ndim == 3:
    X_test = X_test[..., None]

# Align time dimension by cropping/padding to the minimum frames across splits
T_train, T_val, T_test = X_train.shape[2], X_val.shape[2], X_test.shape[2]
T_min = int(min(T_train, T_val, T_test))
if not (T_train == T_val == T_test):
    log(f'Time frames mismatch detected (train={T_train}, val={T_val}, test=

def _pad_or_crop_time(X, T):
```

```

    cur = X.shape[2]
    if cur == T:
        return X
    if cur > T:
        return X[:, :, :T, :]
    # pad at end with zeros
    pad_width = ((0,0),(0,0),(0, T - cur),(0,0))
    return np.pad(np.asarray(X), pad_width, mode='constant')

X_train = _pad_or_crop_time(X_train, T_min)
X_val = _pad_or_crop_time(X_val, T_min)
X_test = _pad_or_crop_time(X_test, T_min)

with open(PROCESSED/'label_encoder.pkl','rb') as f: le = pickle.load(f)
num_classes = len(le.classes_)
y_train_cat = to_categorical(y_train, num_classes)
y_val_cat = to_categorical(y_val, num_classes)
y_test_cat = to_categorical(y_test, num_classes)

log(f'FMA shapes (norm): train={X_train.shape}, val={X_val.shape}, test={X_t

```

2025-08-25 09:02:51.816121: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

2025-08-25 09:02:51.830827: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been registered

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

E0000 00:00:1756105371.846798 49089 cuda_dnn.cc:8579] Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been registered

E0000 00:00:1756105371.851514 49089 cuda_blas.cc:1407] Unable to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has already been registered

W0000 00:00:1756105371.864049 49089 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

W0000 00:00:1756105371.864071 49089 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

W0000 00:00:1756105371.864073 49089 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

W0000 00:00:1756105371.864075 49089 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

2025-08-25 09:02:51.868955: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

[09:02:54] INFO: FMA shapes (norm): train=(1920, 128, 129, 1), val=(640, 128, 129, 1), test=(640, 128, 129, 1) | classes: 8

```

2025-08-25 09:02:54.333326: E external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit: CUDA_ERROR_NO_DEVICE: no CUDA-capable device is detected
2025-08-25 09:02:54.333389: I external/local_xla/xla/stream_executor/cuda/cuda_diagnostics.cc:167] env: CUDA_VISIBLE_DEVICES="-1"
2025-08-25 09:02:54.333400: I external/local_xla/xla/stream_executor/cuda/cuda_diagnostics.cc:170] CUDA_VISIBLE_DEVICES is set to -1 - this hides all GPUs from CUDA
2025-08-25 09:02:54.333407: I external/local_xla/xla/stream_executor/cuda/cuda_diagnostics.cc:178] verbose logging is disabled. Rerun with verbose logging (usually --v=1 or --vmodule=cuda_diagnostics=1) to get more diagnostic output from this module
2025-08-25 09:02:54.333415: I external/local_xla/xla/stream_executor/cuda/cuda_diagnostics.cc:183] retrieving CUDA diagnostic information for host: Jhonny
2025-08-25 09:02:54.333418: I external/local_xla/xla/stream_executor/cuda/cuda_diagnostics.cc:190] hostname: Jhonny
2025-08-25 09:02:54.333635: I external/local_xla/xla/stream_executor/cuda/cuda_diagnostics.cc:197] libcuda reported version is: 575.64.3
2025-08-25 09:02:54.333656: I external/local_xla/xla/stream_executor/cuda/cuda_diagnostics.cc:201] kernel reported version is: 575.64.3
2025-08-25 09:02:54.333658: I external/local_xla/xla/stream_executor/cuda/cuda_diagnostics.cc:291] kernel version seems to match DS0: 575.64.3

```

In [4]: *# Sanity checks: class distributions and label names*

```

log(f'Num classes: {num_classes}')
# Use list() to support both numpy arrays and Python lists without relying on
class_names = list(getattr(le, "classes_", []))
log(f'Class names: {class_names}')

for split_name, y in [ ('train', y_train_cat), ('val', y_val_cat), ('test', y_val_cat) ]:
    counts = np.sum(y, axis=0).astype(int)
    log(f'{split_name} distribution: {counts.tolist()} | total: {int(np.sum(counts))}')

```

```

[09:02:54] INFO: Num classes: 8
[09:02:54] INFO: Class names: [np.str_('Electronic'), np.str_('Experimental'), np.str_('Folk'), np.str_('Hip-Hop'), np.str_('Instrumental'), np.str_('International'), np.str_('Pop'), np.str_('Rock')]
[09:02:54] INFO: train distribution: [240, 240, 240, 240, 240, 240, 240, 240] | total: 1920
[09:02:54] INFO: val distribution: [80, 80, 80, 80, 80, 80, 80, 80] | total: 640
[09:02:54] INFO: test distribution: [80, 80, 80, 80, 80, 80, 80, 80] | total: 640

```

In [5]: *# Optional SpecAugment for training only (frequency/time masking)*

```

import tensorflow as tf
import numpy as np

def spec_augment(mel, freq_mask_param=16, time_mask_param=32, num_masks=2):
    """Apply SpecAugment masks to a single mel-spectrogram tensor.
    mel: float32 tensor [M, T, 1]
    Returns augmented tensor with same shape.
    """

```

```

M = tf.shape(mel)[0]
T = tf.shape(mel)[1]
x = mel
for _ in range(num_masks):
    if FMA_FREQ_MASK_PARAM > 0:
        f = tf.random.uniform([], minval=0, maxval=freq_mask_param + 1,
                                dtype=tf.float32)
        f0 = tf.random.uniform([], minval=0, maxval=tf.maximum(M - f, 1),
                                dtype=tf.float32)
        mask = tf.concat([tf.ones([f0, T, 1]), tf.zeros([f, T, 1]), tf.zeros([M - f0 - f, T, 1])], 0)
        x = x * mask
    if FMA_TIME_MASK_PARAM > 0:
        t = tf.random.uniform([], minval=0, maxval=time_mask_param + 1,
                                dtype=tf.float32)
        t0 = tf.random.uniform([], minval=0, maxval=tf.maximum(T - t, 1),
                                dtype=tf.float32)
        mask = tf.concat([tf.ones([M, t0, 1]), tf.zeros([M, t, 1]), tf.zeros([M, T - t0 - t, 1])], 0)
        x = x * mask
return x

def ds_with_optional_aug(X, y_cat, batch_size, training: bool):
    ds = tf.data.Dataset.from_tensor_slices((X, y_cat))
    if training:
        ds = ds.shuffle(min(10000, len(X)))
    ds = ds.batch(batch_size)
    if training and FMA_SPEC_AUGMENT:
        def _aug(mel, y):
            mel = tf.map_fn(lambda m: spec_augment(m, FMA_FREQ_MASK_PARAM, FMA_TIME_MASK_PARAM), mel, dtype=tf.float32)
            return mel, y
        ds = ds.map(_aug, num_parallel_calls=tf.data.AUTOTUNE)
    ds = ds.prefetch(tf.data.AUTOTUNE)
    return ds

```

```

In [6]: # Assert multi-class splits and show per-class counts (indices -> names)
from collections import Counter

# Use integer labels directly for robust counting
train_counts = Counter(np.asarray(y_train).tolist())
val_counts = Counter(np.asarray(y_val).tolist())
test_counts = Counter(np.asarray(y_test).tolist())

idx_to_name = {i: cls for i, cls in enumerate(le.classes_)}

def fmt_counts(cntr):
    return {idx_to_name.get(i, str(i)): int(n) for i, n in sorted(cntr.items())}

log(f'Train counts: {fmt_counts(train_counts)}')
log(f'Val counts: {fmt_counts(val_counts)}')
log(f'Test counts: {fmt_counts(test_counts)}')

if len(train_counts) < 2:
    raise RuntimeError(
        'Data preparation appears to be single-class on train. Please re-run
    )

```

```
[09:02:54] INFO: Train counts: {np.str_('Electronic'): 240, np.str_('Experimental'): 240, np.str_('Folk'): 240, np.str_('Hip-Hop'): 240, np.str_('Instrumental'): 240, np.str_('International'): 240, np.str_('Pop'): 240, np.str_('Rock'): 240}
[09:02:54] INFO: Val counts: {np.str_('Electronic'): 80, np.str_('Experimental'): 80, np.str_('Folk'): 80, np.str_('Hip-Hop'): 80, np.str_('Instrumental'): 80, np.str_('International'): 80, np.str_('Pop'): 80, np.str_('Rock'): 80}
[09:02:54] INFO: Test counts: {np.str_('Electronic'): 80, np.str_('Experimental'): 80, np.str_('Folk'): 80, np.str_('Hip-Hop'): 80, np.str_('Instrumental'): 80, np.str_('International'): 80, np.str_('Pop'): 80, np.str_('Rock'): 80}
```

```
In [7]: # Define UNet_Audio_Classifier (lean variant matching project baseline)
        from keras import layers, models

        def build_unet_audio_classifier(input_shape, num_classes):
            i = layers.Input(shape=input_shape)
            x = layers.Conv2D(32,3,padding='same',use_bias=False)(i); x = layers.BatchNormalization(x); x = layers.MaxPooling2D(2)(x)
            x = layers.Conv2D(32,3,padding='same',use_bias=False)(x); x = layers.BatchNormalization(x); x = layers.MaxPooling2D(2)(x)
            x = layers.Conv2D(64,3,padding='same',use_bias=False)(x); x = layers.BatchNormalization(x); x = layers.MaxPooling2D(2)(x)
            x = layers.Conv2D(128,3,padding='same',use_bias=False)(x); x = layers.BatchNormalization(x); x = layers.MaxPooling2D(2)(x)
            x = layers.GlobalAveragePooling2D()(x); x = layers.Dropout(0.5)(x)
            o = layers.Dense(num_classes, activation='softmax', dtype='float32')(x)
            return models.Model(i,o,name='UNet_Audio_Classifier')

        input_shape = tuple(int(d) for d in X_train.shape[1:])
        model = build_unet_audio_classifier(input_shape, num_classes)
        model.compile(optimizer=keras.optimizers.Adam(1e-3), loss='categorical_crossentropy')
        model.summary()
```

Model: "UNet_Audio_Classifier"

Layer (type)	Output Shape	Par
input_layer (InputLayer)	(None , 128, 129, 1)	
conv2d (Conv2D)	(None , 128, 129, 32)	
batch_normalization (BatchNormalization)	(None , 128, 129, 32)	
p_re_lu (PReLU)	(None , 128, 129, 32)	
conv2d_1 (Conv2D)	(None , 128, 129, 32)	9
batch_normalization_1 (BatchNormalization)	(None , 128, 129, 32)	
p_re_lu_1 (PReLU)	(None , 128, 129, 32)	
max_pooling2d (MaxPooling2D)	(None , 64, 64, 32)	
conv2d_2 (Conv2D)	(None , 64, 64, 64)	18
batch_normalization_2 (BatchNormalization)	(None , 64, 64, 64)	
p_re_lu_2 (PReLU)	(None , 64, 64, 64)	
max_pooling2d_1 (MaxPooling2D)	(None , 32, 32, 64)	
conv2d_3 (Conv2D)	(None , 32, 32, 128)	73
batch_normalization_3 (BatchNormalization)	(None , 32, 32, 128)	
p_re_lu_3 (PReLU)	(None , 32, 32, 128)	
global_average_pooling2d (GlobalAveragePooling2D)	(None , 128)	
dropout (Dropout)	(None , 128)	
dense (Dense)	(None , 8)	1

Total params: 103,976 (406.16 KB)

Trainable params: 103,464 (404.16 KB)

Non-trainable params: 512 (2.00 KB)

```
In [ ]: # Train with clean logs and checkpoint skip
AUTOTUNE = tf.data.AUTOTUNE

# Use smaller, safer batch size if memory is tight; can be increased later
BATCH_SIZE = FMA_BATCH_SIZE

ckpt_path = MODELS/'UNet_Audio_Classifier_best_FMA.keras'

# Helper to crop/pad along mel/time dims to match target shape
```

```

def _pad_or_crop_axis(X, axis: int, target: int):
    cur = X.shape[axis]
    if cur == target:
        return X
    if cur > target:
        # crop
        slicer = [slice(None)] * X.ndim
        slicer[axis] = slice(0, target)
        return X[tuple(slicer)]
    # pad zeros at end
    pad_width = [(0,0)] * X.ndim
    pad_width[axis] = (0, target - cur)
    return np.pad(np.asarray(X), pad_width, mode='constant')

# Build model or load/resume from checkpoint per toggles
model = None
loaded_ckpt = False
if ckpt_path.exists() and not FMA_FORCE_RETRAIN:
    if FMA_RESUME_FROM_CKPT:
        try:
            model = keras.models.load_model(ckpt_path, compile=False)
            model.compile(optimizer=keras.optimizers.Adam(1e-3), loss=keras.losses.BinaryCrossentropy())
            loaded_ckpt = True
            log(f'Resuming training from checkpoint: {ckpt_path}')
        except Exception as e:
            log(f'Checkpoint load failed ({e}); building a new model', level='error')
    elif SKIP_TRAIN_IF_CKPT:
        try:
            model = keras.models.load_model(ckpt_path, compile=False)
            model.compile(optimizer=keras.optimizers.Adam(1e-3), loss=keras.losses.BinaryCrossentropy())
            loaded_ckpt = True
            log(f'Loaded existing checkpoint: {ckpt_path} - will skip training')
        except Exception as e:
            log(f'Checkpoint load failed ({e}); building a new model', level='error')

if model is None:
    # Build fresh model matching current data shape
    input_shape = tuple(int(d) for d in X_train.shape[1:])
    model = build_unet_audio_classifier(input_shape, num_classes)
    model.compile(optimizer=keras.optimizers.Adam(1e-3), loss=keras.losses.BinaryCrossentropy())

# Align data tensors to model's expected input shape
expected = model.input_shape # (None, M, T, C)
M_tgt = int(expected[1]) if expected[1] is not None else X_train.shape[1]
T_tgt = int(expected[2]) if expected[2] is not None else X_train.shape[2]
C_tgt = int(expected[3]) if expected[3] is not None else X_train.shape[3]

M_cur, T_cur, C_cur = X_train.shape[1], X_train.shape[2], X_train.shape[3]
if (M_cur, T_cur, C_cur) != (M_tgt, T_tgt, C_tgt):
    log(f'Model expects (M,T,C)=({M_tgt},{T_tgt},{C_tgt}); data has ({M_cur},{T_cur},{C_cur})')
    # Ensure channel dim
    if X_train.ndim == 3:
        X_train = X_train[..., None]
        X_val = X_val[..., None]
        X_test = X_test[..., None]
    # Adjust mel bins and time frames

```

```

    if X_train.shape[1] != M_tgt:
        X_train = _pad_or_crop_axis(X_train, axis=1, target=M_tgt)
        X_val = _pad_or_crop_axis(X_val, axis=1, target=M_tgt)
        X_test = _pad_or_crop_axis(X_test, axis=1, target=M_tgt)
    if X_train.shape[2] != T_tgt:
        X_train = _pad_or_crop_axis(X_train, axis=2, target=T_tgt)
        X_val = _pad_or_crop_axis(X_val, axis=2, target=T_tgt)
        X_test = _pad_or_crop_axis(X_test, axis=2, target=T_tgt)
    log(f'Adjusted shapes: train={X_train.shape}, val={X_val.shape}, test={X_test.shape}')

# Build tf.data pipelines after finalizing shapes
train_ds = ds_with_optional_aug(X_train, y_train_cat, BATCH_SIZE, training=True)
val_ds = ds_with_optional_aug(X_val, y_val_cat, BATCH_SIZE, training=False)
test_ds = ds_with_optional_aug(X_test, y_test_cat, BATCH_SIZE, training=False)

# Optional class weighting to handle class imbalance
from sklearn.utils.class_weight import compute_class_weight
classes_idx = np.arange(num_classes)
class_weights_vec = compute_class_weight(class_weight='balanced', classes=classes_idx, y_train=y_train_cat)
CLASS_WEIGHT = {int(i): float(w) for i, w in zip(classes_idx, class_weights_vec)}
log(f'Class weights: {CLASS_WEIGHT}')

cb = [
    callbacks.EarlyStopping(monitor='val_accuracy', patience=FMA_PATIENCE, restore_best_weights=True),
    callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=max_patience, restore_best_weights=True),
    callbacks.ModelCheckpoint(ckpt_path, monitor='val_accuracy', save_best_only=True)
]

# Decide training path
history = None
if FMA_FORCE_RETRAIN:
    log('FORCE_RETRAIN=1 → training from scratch, ignoring checkpoint')
    history = model.fit(train_ds, validation_data=val_ds, epochs=FMA_EPOCHS, callbacks=cb)
elif FMA_RESUME_FROM_CKPT and loaded_ckpt:
    log('RESUME_FROM_CKPT=1 → continuing training from checkpoint')
    history = model.fit(train_ds, validation_data=val_ds, epochs=FMA_EPOCHS, callbacks=cb)
elif SKIP_TRAIN_IF_CKPT and loaded_ckpt:
    log('Training skipped due to existing checkpoint.')
else:
    log('No checkpoint scenario → training from scratch')
    history = model.fit(train_ds, validation_data=val_ds, epochs=FMA_EPOCHS, callbacks=cb)

log('Evaluating on test set...')

test_loss, test_acc = model.evaluate(test_ds, verbose=0)
log(f'FMA Test Accuracy: {test_acc}')

best_val = float(np.max(history.history.get('val_accuracy', [0]))) if history else 0.0

pd.DataFrame([
    {
        'Model': 'UNet_Audio_Classifier', 'Dataset': 'FMA_SMALL',
        'Best_Val_Accuracy': best_val,
        'Test_Accuracy': float(test_acc), 'Epochs_Run': int(len(history.history.get('val_accuracy', [])))
    }
])

```

```
]).to_csv(REPORTS/'training_summary_FMA.csv', index=False)
log(f'Saved: {REPORTS/"training_summary_FMA.csv"}')
```

```
[09:02:55] INFO: Class weights: {0: 1.0, 1: 1.0, 2: 1.0, 3: 1.0, 4: 1.0, 5:
1.0, 6: 1.0, 7: 1.0}
[09:02:55] INFO: No checkpoint scenario → training from scratch
Epoch 1/80
60/60 ██████████ 39s 611ms/step - accuracy: 0.2401 - loss: 2.0574
- val_accuracy: 0.1797 - val_loss: 2.1565 - learning_rate: 0.0010
Epoch 2/80
60/60 ██████████ 35s 583ms/step - accuracy: 0.2829 - loss: 1.9330
- val_accuracy: 0.1437 - val_loss: 2.1988 - learning_rate: 0.0010
Epoch 3/80
60/60 ██████████ 35s 589ms/step - accuracy: 0.3089 - loss: 1.9122
- val_accuracy: 0.1703 - val_loss: 2.2894 - learning_rate: 0.0010
Epoch 4/80
60/60 ██████████ 35s 581ms/step - accuracy: 0.3680 - loss: 1.7648
- val_accuracy: 0.1547 - val_loss: 2.3295 - learning_rate: 0.0010
Epoch 5/80
60/60 ██████████ 36s 600ms/step - accuracy: 0.3830 - loss: 1.7406
- val_accuracy: 0.2141 - val_loss: 2.3082 - learning_rate: 0.0010
Epoch 6/80
60/60 ██████████ 36s 607ms/step - accuracy: 0.4052 - loss: 1.6829
- val_accuracy: 0.1922 - val_loss: 2.9232 - learning_rate: 0.0010
Epoch 7/80
60/60 ██████████ 36s 594ms/step - accuracy: 0.4020 - loss: 1.6623
- val_accuracy: 0.2297 - val_loss: 2.5599 - learning_rate: 0.0010
Epoch 8/80
60/60 ██████████ 36s 601ms/step - accuracy: 0.4100 - loss: 1.6604
- val_accuracy: 0.3281 - val_loss: 2.1640 - learning_rate: 0.0010
Epoch 9/80
60/60 ██████████ 36s 590ms/step - accuracy: 0.4656 - loss: 1.6004
- val_accuracy: 0.3875 - val_loss: 1.9645 - learning_rate: 2.0000e-04
Epoch 10/80
60/60 ██████████ 35s 586ms/step - accuracy: 0.4442 - loss: 1.5712
- val_accuracy: 0.3578 - val_loss: 2.0681 - learning_rate: 2.0000e-04
Epoch 11/80
60/60 ██████████ 35s 577ms/step - accuracy: 0.4814 - loss: 1.5289
- val_accuracy: 0.3703 - val_loss: 2.0962 - learning_rate: 2.0000e-04
Epoch 12/80
60/60 ██████████ 36s 602ms/step - accuracy: 0.4670 - loss: 1.5338
- val_accuracy: 0.4016 - val_loss: 1.9195 - learning_rate: 2.0000e-04
Epoch 13/80
16/60 ██████████ 23s 541ms/step - accuracy: 0.5282 - loss: 1.4428
```

```
In [ ]: # After training: save classification report and confusion matrix
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Get predictions
y_pred = model.predict(test_ds, verbose=0)
y_pred_idx = np.argmax(y_pred, axis=1)
y_true_idx = np.argmax(y_test_cat, axis=1)

# Classification report
```

```

report = classification_report(y_true_idx, y_pred_idx, target_names=le.classes_)
with open(REPORTS/'classification_report_UNet_Audio_Classifier_FMA.txt', 'w') as f:
    f.write(report)
log(f'Saved: {REPORTS/"classification_report_UNet_Audio_Classifier_FMA.txt"}')

# Confusion matrix
cm = confusion_matrix(y_true_idx, y_pred_idx)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=le.classes_, yticklabels=le.classes_)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix – UNet_Audio_Classifier (FMA)')
plt.tight_layout()
fig_path = REPORTS/'confusion_matrix_UNet_Audio_Classifier_FMA.png'
plt.savefig(fig_path)
plt.close()
log(f'Saved: {fig_path}')

```

[19:18:09] INFO: Saved: /home/alepot55/Desktop/projects/naml_project/reports/classification_report_UNet_Audio_Classifier_FMA.txt

[19:18:09] INFO: Saved: /home/alepot55/Desktop/projects/naml_project/reports/confusion_matrix_UNet_Audio_Classifier_FMA.png